

Part A

Count / Frequency based Naive Bayes Classifier

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer,
CountVectorizer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score
)

# =====
# TODO: Students must implement the following steps:
# 1. Complete the fit method in NaiveBayesClassifier (4 TODOs for log
# prior and log likelihood calculation).
# 2. Complete the predict method in NaiveBayesClassifier (2 TODOs for
# log probability accumulation and final argmax).
# 3. Complete the data loading calls in Section 2.
# 4. Initialize CountVectorizer with proper parameters in Section 3a.
# 5. Complete the feature transformation (fit_transform and transform)
# in Section 3a.
# 6. Initialize and fit the custom nb_model in Section 3b.
# 7. Use the fitted nb_model to generate predictions in Section 4.
# =====

# Data loading function (DO NOT CHANGE)
def load_pubmed_rct_file(filepath):
    """
    Reads a .txt file from the PubMed 20k RCT dataset.
    Returns a DataFrame with 'label' and 'sentence'.
    """
    labels, sentences = [], []
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f:
            line = line.strip()
            if not line or '\t' not in line:
```

```

        continue
    label, sent = line.split('\t', maxsplit=1)
    labels.append(label)
    sentences.append(sent)
return pd.DataFrame({'label': labels, 'sentence': sentences})

# Implementing Multinomial Naive Bayes from scratch
class NaiveBayesClassifier:
    """
    Multinomial Naive Bayes Classifier implemented from scratch.
    It is suitable for both Count and TF-IDF features.
    """
    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.class_priors = {}
        self.feature_log_probs = {}
        self.classes = None
        self.vocabulary_size = 0

    def fit(self, X_counts, y):
        y_array = y.to_numpy()
        self.classes = np.unique(y_array)
        self.vocabulary_size = X_counts.shape[1]

        for c in self.classes:
            X_c = X_counts[y_array == c]

            # Calculate the log prior and store it in
            self.class_priors[c]
            # Calculate Class Prior P(C): log(P(C))
            # P(C) = (Number of samples in class c) / (Total number of
            samples)
            self.class_priors[c] = np.log(X_c.shape[0] /
            X_counts.shape[0])

            feature_sum = X_c.sum(axis=0).A1
            total_mass = np.sum(feature_sum)

            # Apply Laplace smoothing (additive smoothing, alpha=1.0
            default):
            #  $P(w_i | C) = (\text{count}(w_i, C) + \alpha) / (\text{total\_words\_in\_C} + \alpha * \text{vocab\_size})$ 
            # Calculate the numerator (with Laplace smoothing)
            numerator = feature_sum + self.alpha

            # Calculate the denominator (with Laplace smoothing)
            denominator = total_mass + self.alpha *
            self.vocabulary_size

```

```

        # Calculate the log likelihood (log(numerator /
denominator))
        self.feature_log_probs[c] = np.log(numerator /
denominator)

    def predict(self, X_counts):
        y_pred = []
        for i in range(X_counts.shape[0]):
            scores = {}

            x_i = X_counts.getrow(i)

            for c in self.classes:
                log_prob = self.class_priors[c]
                log_likelihoods = self.feature_log_probs[c]

                non_zero_indices = x_i.indices
                non_zero_data = x_i.data

                # Complete the log probability calculation for the
likelihood term
                # Add log likelihoods contribution (Log-Sum Trick):
                # log_prob += sum(count(w_i) * log(P(w_i|C)))

                log_prob += np.sum(non_zero_data *
log_likelihoods[non_zero_indices])
                scores[c] = log_prob

            # Find the key (class label) with the maximum score
            predicted_class = max(scores, key=scores.get)

            y_pred.append(predicted_class)

        # Return the final predictions array
        return np.array(y_pred)

# Load and Prepare Data (DO NOT CHANGE)
dir_path = './'
try:
    train_df = load_pubmed_rct_file(os.path.join(dir_path,
'train.txt'))
    dev_df    = load_pubmed_rct_file(os.path.join(dir_path, 'dev.txt'))
    test_df   = load_pubmed_rct_file(os.path.join(dir_path,
'test.txt'))

    print(f"Train samples: {len(train_df)}")
    print(f"Dev   samples: {len(dev_df)}")
    print(f"Test  samples: {len(test_df)}")

```

```

X_train, y_train = train_df['sentence'], train_df['label']
X_dev, y_dev = dev_df['sentence'], dev_df['label']
X_test, y_test = test_df['sentence'], test_df['label']
target_names = sorted(y_train.unique())
print(f"Classes: {target_names}")

except FileNotFoundError as e:
    print(f"Error: Dataset file not found. Please ensure the files are
uploaded.")
    X_train, y_train = pd.Series([]), pd.Series([])
    X_test, y_test = pd.Series([]), pd.Series([])
    target_names = []

Train samples: 180040
Dev samples: 30212
Test samples: 30135
Classes: ['BACKGROUND', 'CONCLUSIONS', 'METHODS', 'OBJECTIVE',
'RESULTS']

# Feature Extraction and Custom Model Training
if X_train is not None and len(X_train) > 0:

    # Initialize and fit the CountVectorizer for count-based features
    count_vectorizer = CountVectorizer(
        lowercase=True,
        strip_accents='unicode',
        stop_words='english',
        # Set appropriate ngram_range
        ngram_range=(1, 2),
        # Set appropriate min_df
        min_df=3
    )

    print("Fitting Count Vectorizer and transforming training
data...")
    # Fit the vectorizer on X_train and transform
    X_train_counts = count_vectorizer.fit_transform(X_train)
    if X_train_counts is not None:
        print(f"Vocabulary size: {X_train_counts.shape[1]}")

    print("Transforming test data...")
    # Transform X_test using the fitted vectorizer
    X_test_counts = count_vectorizer.transform(X_test)

    # Train Custom Naive Bayes Classifier
    print("\nTraining the Custom Naive Bayes Classifier (from
scratch)...")

    # Initialize the custom NaiveBayesClassifier

```

```

nb_model = NaiveBayesClassifier(alpha=1.0)

# Fit the model using X_train_counts and y_train
nb_model.fit(X_train_counts, y_train)
print("Training complete.")

else:
    print("Skipping feature extraction and training: Training data is
empty or not loaded.")

Fitting Count Vectorizer and transforming training data...
Vocabulary size: 166869
Transforming test data...
Vocabulary size: 166869
Transforming test data...

Training the Custom Naive Bayes Classifier (from scratch)...

Training the Custom Naive Bayes Classifier (from scratch)...
Training complete.
Training complete.

# Predict and evaluate on test set
print("\n=== Test Set Evaluation (Custom Count-Based Naive Bayes)
===")

# Predict y_test_pred using X_test_counts
y_test_pred = nb_model.predict(X_test_counts)

if y_test_pred is not None:
    print(f"Accuracy: {accuracy_score(y_test, y_test_pred):.4f}")
    print(classification_report(y_test, y_test_pred,
target_names=target_names))
    test_f1 = f1_score(y_test, y_test_pred, average='macro')
    print(f"Macro-averaged F1 score: {test_f1:.4f}")
else:
    print("Prediction step failed or incomplete.")

```

=== Test Set Evaluation (Custom Count-Based Naive Bayes) ===

Accuracy: 0.7537

Accuracy: 0.7537

	precision	recall	f1-score	support
BACKGROUND	0.55	0.57	0.56	3621
CONCLUSIONS	0.62	0.70	0.66	4571
METHODS	0.82	0.87	0.85	9897
OBJECTIVE	0.57	0.48	0.52	2333
RESULTS	0.88	0.79	0.83	9713
accuracy			0.75	30135

macro avg	0.69	0.68	0.68	30135
weighted avg	0.76	0.75	0.75	30135

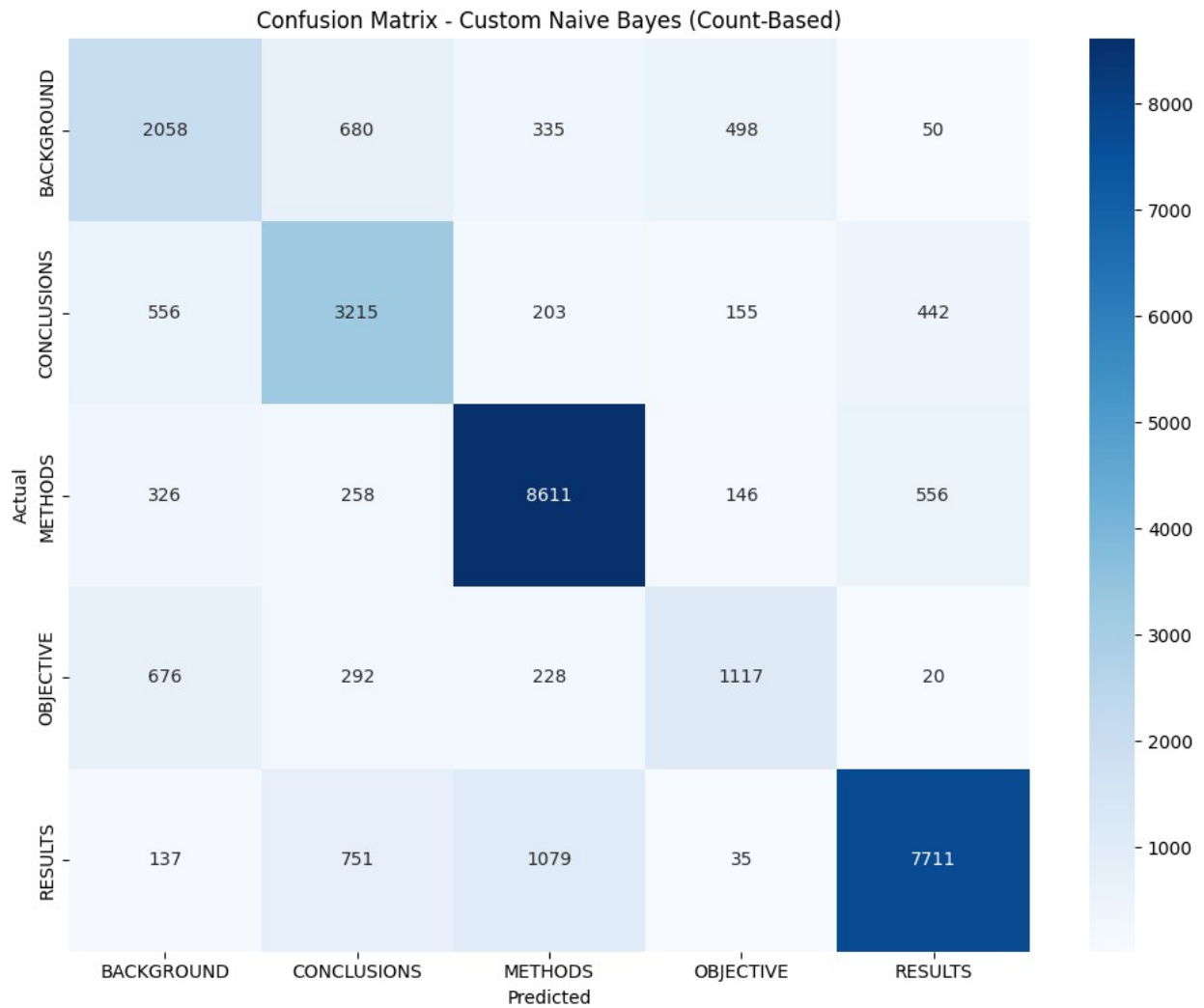
Macro-averaged F1 score: 0.6836

	precision	recall	f1-score	support
BACKGROUND	0.55	0.57	0.56	3621
CONCLUSIONS	0.62	0.70	0.66	4571
METHODS	0.82	0.87	0.85	9897
OBJECTIVE	0.57	0.48	0.52	2333
RESULTS	0.88	0.79	0.83	9713

accuracy			0.75	30135
macro avg	0.69	0.68	0.68	30135
weighted avg	0.76	0.75	0.75	30135

Macro-averaged F1 score: 0.6836

```
# Confusion Matrix on test set
# Use the confusion_matrix, matplotlib, and seaborn libraries to
generate
# a visual confusion matrix (heatmap) for the predicted results.
if y_test_pred is not None:
    cm = confusion_matrix(y_test, y_test_pred, labels=target_names)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=target_names, yticklabels=target_names)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.title('Confusion Matrix - Custom Naive Bayes (Count-Based)')
    plt.tight_layout()
    plt.show()
```



Part B

TF-IDF score based Classifier

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import (
    accuracy_score,
    classification_report,
```

```

        confusion_matrix,
        f1_score
    )

# =====
# TODO: Implement the following steps:
# 1. Define the initial `pipeline` combining TfidfVectorizer and
# MultinomialNB with default parameters.
# 2. Train the initial pipeline on the training data (X_train,
# y_train).
# 3. Predict and evaluate the performance of the initial model on the
# test data (X_test, y_test).
# 4. Define the `param_grid` for hyperparameter tuning.
# 5. Initialize `GridSearchCV` using the pipeline, parameter grid, and
# appropriate cross-validation settings.
# 6. Fit the Grid Search object using the development data (X_dev,
# y_dev).
# 7. Print the `best_params_` and `best_score_` found by the grid
# search.
# =====

# Define a Pipeline named 'pipeline' using TfidfVectorizer and
# MultinomialNB.
# Use standard initial parameters
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(lowercase=True, strip_accents='unicode',
stop_words='english')),
    ('nb', MultinomialNB())
])

# Train the initial pipeline on the training set
print("Training initial Naive Bayes pipeline...")
pipeline.fit(X_train, y_train)
print("Training complete.")

# Predict and evaluate on test set
# Predict y_test_pred and calculate metrics
print("\n=== Test Set Evaluation (Initial Sklearn Model) ===")
y_test_pred = pipeline.predict(X_test)
if y_test_pred is not None:
    print(f"Accuracy: {accuracy_score(y_test, y_test_pred):.4f}")
    print(classification_report(y_test, y_test_pred,
target_names=target_names))
    print(f"Macro-averaged F1 score: {f1_score(y_test, y_test_pred,
average='macro'):.4f}")
    pass
else:
    print("Initial model evaluation skipped: Predictions not

```

```

available.")

# Hyperparameter Tuning using GridSearchCV

# Define the parameter grid 'param_grid' to tune both TF-IDF and NB
parameters.
param_grid = {
    'tfidf__ngram_range': [(1, 1), (1, 2), (1, 3)],
    'tfidf__min_df': [2, 3, 5],
    'nb__alpha': [0.1, 0.5, 1.0, 2.0]
}

# Initialize GridSearchCV using the pipeline and param_grid.
# Ensure cv=3 and scoring='f1_macro' are used.
grid = GridSearchCV(pipeline, param_grid, cv=3, scoring='f1_macro',
n_jobs=-1, verbose=1)

print("\nStarting Hyperparameter Tuning on Development Set...")
# Fit the GridSearchCV object using the development data.
grid.fit(X_dev, y_dev)
print("Grid search complete.")

if grid is not None and hasattr(grid, 'best_params_'):
    # Print the best parameters and the corresponding best cross-
    validation score.
    print(f"\nBest parameters: {grid.best_params_}")
    print(f"Best cross-validation F1 score: {grid.best_score_:.4f}")

    # Evaluate best model on test set
    print("\n=== Test Set Evaluation (Tuned Model) ===")
    y_test_pred_tuned = grid.predict(X_test)
    print(f"Accuracy: {accuracy_score(y_test,
y_test_pred_tuned):.4f}")
    print(classification_report(y_test, y_test_pred_tuned,
target_names=target_names))
    print(f"Macro-averaged F1 score: {f1_score(y_test,
y_test_pred_tuned, average='macro'):.4f}")
else:
    print("Hyperparameter tuning skipped: Grid Search object not
initialized or fitted.")

Training initial Naive Bayes pipeline...
Training complete.

=== Test Set Evaluation (Initial Sklearn Model) ===
Training complete.

=== Test Set Evaluation (Initial Sklearn Model) ===

```

Accuracy: 0.6996

Accuracy: 0.6996

	precision	recall	f1-score	support
BACKGROUND	0.61	0.37	0.46	3621
CONCLUSIONS	0.61	0.55	0.57	4571
METHODS	0.68	0.88	0.77	9897
OBJECTIVE	0.72	0.09	0.16	2333
RESULTS	0.77	0.85	0.81	9713
accuracy			0.70	30135
macro avg	0.68	0.55	0.56	30135
weighted avg	0.69	0.70	0.67	30135

Macro-averaged F1 score: 0.5555

Starting Hyperparameter Tuning on Development Set...

Fitting 3 folds for each of 36 candidates, totalling 108 fits

	precision	recall	f1-score	support
BACKGROUND	0.61	0.37	0.46	3621
CONCLUSIONS	0.61	0.55	0.57	4571
METHODS	0.68	0.88	0.77	9897
OBJECTIVE	0.72	0.09	0.16	2333
RESULTS	0.77	0.85	0.81	9713
accuracy			0.70	30135
macro avg	0.68	0.55	0.56	30135
weighted avg	0.69	0.70	0.67	30135

Macro-averaged F1 score: 0.5555

Starting Hyperparameter Tuning on Development Set...

Fitting 3 folds for each of 36 candidates, totalling 108 fits

Grid search complete.

Best parameters: {'nb__alpha': 0.1, 'tfidf__min_df': 5, 'tfidf__ngram_range': (1, 3)}

Best cross-validation F1 score: 0.6308

=== Test Set Evaluation (Tuned Model) ===

Grid search complete.

Best parameters: {'nb__alpha': 0.1, 'tfidf__min_df': 5, 'tfidf__ngram_range': (1, 3)}

Best cross-validation F1 score: 0.6308

=== Test Set Evaluation (Tuned Model) ===

Accuracy: 0.7198

Accuracy: 0.7198

	precision	recall	f1-score	support
BACKGROUND	0.54	0.44	0.48	3621
CONCLUSIONS	0.58	0.63	0.61	4571
METHODS	0.77	0.86	0.81	9897
OBJECTIVE	0.54	0.37	0.44	2333
RESULTS	0.82	0.81	0.82	9713
accuracy			0.72	30135
macro avg	0.65	0.62	0.63	30135
weighted avg	0.71	0.72	0.71	30135
Macro-averaged F1 score: 0.6309				
	precision	recall	f1-score	support
BACKGROUND	0.54	0.44	0.48	3621
CONCLUSIONS	0.58	0.63	0.61	4571
METHODS	0.77	0.86	0.81	9897
OBJECTIVE	0.54	0.37	0.44	2333
RESULTS	0.82	0.81	0.82	9713
accuracy			0.72	30135
macro avg	0.65	0.62	0.63	30135
weighted avg	0.71	0.72	0.71	30135
Macro-averaged F1 score: 0.6309				

Part C

Bayes Optimal Classifier

Part C Draft

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.tree import DecisionTreeClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import FunctionTransformer
from sklearn.naive_bayes import MultinomialNB
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import accuracy_score, f1_score,
```

```

classification_report, confusion_matrix
from sklearn.model_selection import train_test_split # Used for  $P(h|D)$ 
calculation
import time

# =====
# TODO: Implement the following steps:
# 1. Train all five hypotheses on the sampled training data.
# 2. Compute the Posterior Weights  $P(h_i | D)$  using a validation
split.
# 3. Fit the VotingClassifier using the sampled training data.
# 4. Make final predictions and evaluate the BOC performance on the
test data.
# =====

# Dynamic Data Sampling (OPTIMIZED FOR FASTER EXECUTION)
BASE_SAMPLE_SIZE = 5000 # Reduced from 10000 for faster training

# Prompt the user for their full SRN
FULL_SRN = input("Please enter your full SRN (e.g., PES1UG22CS345): ")

try:
    # Extract the last three characters and convert to integer
    if len(FULL_SRN) >= 3:
        srn_suffix_str = FULL_SRN[-3:]
        srn_value = int(srn_suffix_str)
    else:
        # Fallback if input is too short
        raise ValueError("SRN too short.")
except (ValueError, IndexError, TypeError):
    # Fallback if SRN is not entered or format is incorrect
    print("WARNING: SRN input failed or format is incorrect. Using
base size.")
    srn_value = 0

# Calculate the final sample size: 5000 + (last three SRN digits / 2)
for balance
SAMPLE_SIZE = BASE_SAMPLE_SIZE + (srn_value // 2)

print(f"Using dynamic sample size: {SAMPLE_SIZE}")

# Placeholder initialization in case data wasn't loaded in the
environment
if 'X_train' not in locals() or len(X_train) == 0:
    print("Warning: Training data not found. Using small placeholder
data.")
    X_train = pd.Series(["sample text one"] * 11000)
    y_train = pd.Series(["BACKGROUND"] * 5000 + ["METHODS"] * 6000)
    X_test = pd.Series(["test text one", "test text two"])
    y_test = pd.Series(["BACKGROUND", "METHODS"])

```

```

    target_names = ["BACKGROUND", "CONCLUSIONS", "METHODS",
"OBJECTIVE", "RESULTS"]

# Use stratified sampling to ensure all classes are represented
effective_sample_size = min(SAMPLE_SIZE, len(X_train))
if effective_sample_size < len(X_train):
    # Stratified sampling to maintain class distribution
    from sklearn.model_selection import train_test_split
    X_train_sampled, _, y_train_sampled, _ = train_test_split(
        X_train, y_train,
        train_size=effective_sample_size,
        random_state=42,
        stratify=y_train
    )
else:
    X_train_sampled = X_train
    y_train_sampled = y_train
print(f"Actual sampled training set size used:
{len(X_train_sampled)}")
print(f"Classes in sample: {sorted(y_train_sampled.unique())}")

# Base TF-IDF parameters (DO NOT CHANGE)
tfidf_params = {
    'lowercase': True,
    'strip_accents': 'unicode',
    'stop_words': 'english',
    'ngram_range': (1, 1), # Using unigrams only to keep feature space
small for diverse models
    'min_df': 5
}

# Define the five diverse hypotheses/pipelines (DO NOT CHANGE)

# H1: Multinomial Naive Bayes
h1_nb = Pipeline([
    ('tfidf', TfidfVectorizer(**tfidf_params)),
    ('clf', MultinomialNB(alpha=1.0, fit_prior=False))
])

# H2: Logistic Regression
h2_lr = Pipeline([
    ('tfidf', TfidfVectorizer(**tfidf_params)),
    ('clf', LogisticRegression(solver='liblinear', multi_class='auto',
max_iter=1000, random_state=42))
])

# H3: Random Forest Classifier (OPTIMIZED)
h3_rf = Pipeline([
    ('tfidf', TfidfVectorizer(**tfidf_params)),

```

```

        ('clf', RandomForestClassifier(n_estimators=20, max_depth=8,
random_state=42, n_jobs=-1))
    ])

# H4: Decision Tree Classifier (OPTIMIZED)
h4_dt = Pipeline([
    ('tfidf', TfidfVectorizer(**tfidf_params)),
    ('clf', DecisionTreeClassifier(max_depth=8, random_state=42))
])

# H5: K-Nearest Neighbors (OPTIMIZED)
h5_knn = Pipeline([
    ('tfidf', TfidfVectorizer(**tfidf_params)),
    ('clf', KNeighborsClassifier(n_neighbors=3, n_jobs=-1))
])

hypotheses = [h1_nb, h2_lr, h3_rf, h4_dt, h5_knn]
hypothesis_names = ['NaiveBayes', 'LogisticRegression',
'RandomForest', 'DecisionTree', 'KNN']

# Training and BOC Implementation (STUDENT TASK)

# Train all five hypotheses on X_train_sampled and y_train_sampled
using a for loop.
print("\nTraining all base models...")
print("=" * 60)

for i, (name, model) in enumerate(zip(hypothesis_names, hypotheses)):
    start_time = time.time()
    print(f"\n[{i+1}/{len(hypotheses)}] Training {name}...")
    model.fit(X_train_sampled, y_train_sampled)
    elapsed = time.time() - start_time
    print(f"    ✓ Done in {elapsed:.1f}s")

print("\n" + "=" * 60)
print("All base models trained successfully!")
print("=" * 60)

# Implement the Posterior Weight Calculation ( $P(h_i | D)$ ).
# This requires splitting X_train_sampled into a small
train_sub/val_sub set
# and calculating the validation log-likelihood for each model.
Normalize these to get posterior_weights.

# Split for validation (smaller validation set for speed)
X_train_sub, X_val_sub, y_train_sub, y_val_sub = train_test_split(
    X_train_sampled, y_train_sampled, test_size=0.15, random_state=42,
    stratify=y_train_sampled

```

```

)

print("\nCalculating posterior weights...")
print("=" * 60)
log_likelihoods = []

for i, (name, model) in enumerate(zip(hypothesis_names, hypotheses)):
    print(f"[{i+1}/{len(hypotheses)}] Computing weight for {name}...")
    # Get probability predictions on validation set
    try:
        y_proba = model.predict_proba(X_val_sub)
        # Calculate log-likelihood
        y_val_indices = [list(model.classes_).index(label) for label
in y_val_sub]
        log_likelihood =
np.sum(np.log(y_proba[np.arange(len(y_val_sub)), y_val_indices] + 1e-
10))
        log_likelihoods.append(log_likelihood)
        print(f"    → log-likelihood: {log_likelihood:.2f}")
    except Exception as e:
        print(f"    → FAILED: {e}")
        log_likelihoods.append(-1e6) # Low weight for failed models

# Convert log-likelihoods to probabilities using softmax
log_likelihoods = np.array(log_likelihoods)
# Subtract max for numerical stability
log_likelihoods = log_likelihoods - np.max(log_likelihoods)
posterior_weights = np.exp(log_likelihoods) /
np.sum(np.exp(log_likelihoods))

print("\n" + "=" * 60)
print("POSTERIOR WEIGHTS (P(h|D)):")
print("-" * 60)
for name, weight in zip(hypothesis_names, posterior_weights):
    bar_length = int(weight * 50)
    bar = "█" * bar_length + "░" * (50 - bar_length)
    print(f"    {name:20s} | {bar} {weight:.4f}")
print("=" * 60)

posterior_weights = posterior_weights.tolist()

# Implement and Evaluate the Bayes Optimal Classifier
estimators = list(zip(hypothesis_names, hypotheses))

# BOC is approximated using soft voting with posterior weights
boc_soft_voter = VotingClassifier(
    estimators=estimators,
    voting='soft',
    weights=posterior_weights,

```

```

        n_jobs=-1
    )

    print("\nFitting the VotingClassifier (BOC approximation)...")
    print("=" * 60)
    start_time = time.time()

    # Fit the VotingClassifier using the full sampled training data
    (X_train_sampled, y_train_sampled)
    boc_soft_voter.fit(X_train_sampled, y_train_sampled)

    elapsed = time.time() - start_time
    print(f"✓ VotingClassifier fitted successfully in {elapsed:.1f}s")
    print("=" * 60)

    # Make the final BOC prediction on the test set
    print("\nPredicting on test set...")
    start_time = time.time()

    # Predict y_pred using X_test, and then calculate and visualize
    # evaluation metrics.
    y_pred = boc_soft_voter.predict(X_test)

    elapsed = time.time() - start_time
    print(f"✓ Predictions complete in {elapsed:.1f}s")

    # Final Evaluation (STUDENT TASK)
    print("\n=== Final Evaluation: Bayes Optimal Classifier (Soft Voting)
    ===")

    if y_pred is not None:
        # Calculate metrics
        accuracy = accuracy_score(y_test, y_pred)
        f1_macro = f1_score(y_test, y_pred, average='macro')

        print(f"Accuracy: {accuracy:.4f}")
        print(f"Macro-averaged F1 score: {f1_macro:.4f}")
        print("\nClassification Report:")
        print(classification_report(y_test, y_pred,
        target_names=target_names))

        # Generate and visualize the Confusion Matrix (heatmap) for the
        # BOC predictions.
        cm = confusion_matrix(y_test, y_pred, labels=target_names)
        plt.figure(figsize=(10, 8))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
        xticklabels=target_names, yticklabels=target_names)
        plt.xlabel('Predicted')

```

```

plt.ylabel('Actual')
plt.title('Confusion Matrix - Bayes Optimal Classifier (Soft
Voting)')
plt.tight_layout()
plt.show()
else:
    print("Evaluation skipped: Predictions not generated.")

```

Using dynamic sample size: 5032

Actual sampled training set size used: 5032

Classes in sample: ['BACKGROUND', 'CONCLUSIONS', 'METHODS',
'OBJECTIVE', 'RESULTS']

Training all base models...

=====

[1/5] Training NaiveBayes...

✓ Done in 0.1s

[2/5] Training LogisticRegression...

Actual sampled training set size used: 5032

Classes in sample: ['BACKGROUND', 'CONCLUSIONS', 'METHODS',
'OBJECTIVE', 'RESULTS']

Training all base models...

=====

[1/5] Training NaiveBayes...

✓ Done in 0.1s

[2/5] Training LogisticRegression...

C:\Users\aritr\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_logistic.py:1272: FutureWarning: 'multi_class' was deprecated in version 1.5 and will be removed in 1.8. From then on, it will always use 'multinomial'. Leave it to its default value to avoid this warning.

warnings.warn(

C:\Users\aritr\AppData\Roaming\Python\Python313\site-packages\sklearn\linear_model_logistic.py:1296: FutureWarning: Using the 'liblinear' solver for multiclass classification is deprecated. An error will be raised in 1.8. Either use another solver which supports the multinomial loss or wrap the estimator in a OneVsRestClassifier to keep applying a one-versus-rest scheme.

warnings.warn(

✓ Done in 0.2s

[3/5] Training RandomForest...

✓ Done in 0.3s

```
[4/5] Training DecisionTree...
✓ Done in 0.2s
```

```
[5/5] Training KNN...
✓ Done in 0.3s
```

```
[4/5] Training DecisionTree...
✓ Done in 0.2s
```

```
[5/5] Training KNN...
✓ Done in 0.1s
```

```
=====
All base models trained successfully!
=====
```

```
Calculating posterior weights...
```

```
=====
[1/5] Computing weight for NaiveBayes...
→ log-likelihood: -600.34
[2/5] Computing weight for LogisticRegression...
→ log-likelihood: -550.74
[3/5] Computing weight for RandomForest...
→ log-likelihood: -967.64
[4/5] Computing weight for DecisionTree...
→ log-likelihood: -915.77
[5/5] Computing weight for KNN...
✓ Done in 0.1s
```

```
=====
All base models trained successfully!
=====
```

```
Calculating posterior weights...
```

```
=====
[1/5] Computing weight for NaiveBayes...
→ log-likelihood: -600.34
[2/5] Computing weight for LogisticRegression...
→ log-likelihood: -550.74
[3/5] Computing weight for RandomForest...
→ log-likelihood: -967.64
[4/5] Computing weight for DecisionTree...
→ log-likelihood: -915.77
[5/5] Computing weight for KNN...
→ log-likelihood: -626.66
```

```
=====
POSTERIOR WEIGHTS (P(h|D)):
-----
NaiveBayes          |
```

LogisticRegression	0.0000
RandomForest	1.0000
DecisionTree	0.0000
KNN	0.0000

Fitting the VotingClassifier (BOC approximation)...

→ log-likelihood: -626.66

POSTERIOR WEIGHTS (P(h|D)):

NaiveBayes	0.0000
LogisticRegression	1.0000
RandomForest	0.0000
DecisionTree	0.0000
KNN	0.0000

Fitting the VotingClassifier (BOC approximation)...

✓ VotingClassifier fitted successfully in 0.4s

Predicting on test set...

✓ VotingClassifier fitted successfully in 0.4s

Predicting on test set...

✓ Predictions complete in 7.4s

=== Final Evaluation: Bayes Optimal Classifier (Soft Voting) ===

Accuracy: 0.6787

Macro-averaged F1 score: 0.5688

Classification Report:

✓ Predictions complete in 7.4s

=== Final Evaluation: Bayes Optimal Classifier (Soft Voting) ===

Accuracy: 0.6787
Macro-averaged F1 score: 0.5688

Classification Report:

	precision	recall	f1-score	support
BACKGROUND	0.53	0.35	0.42	3621
CONCLUSIONS	0.58	0.47	0.52	4571
METHODS	0.67	0.87	0.76	9897
OBJECTIVE	0.63	0.26	0.36	2333
RESULTS	0.76	0.80	0.78	9713

accuracy			0.68	30135
macro avg	0.64	0.55	0.57	30135
weighted avg	0.67	0.68	0.66	30135

	precision	recall	f1-score	support
BACKGROUND	0.53	0.35	0.42	3621
CONCLUSIONS	0.58	0.47	0.52	4571
METHODS	0.67	0.87	0.76	9897
OBJECTIVE	0.63	0.26	0.36	2333
RESULTS	0.76	0.80	0.78	9713

accuracy			0.68	30135
macro avg	0.64	0.55	0.57	30135
weighted avg	0.67	0.68	0.66	30135

